Assignment-11.5

Task Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty
methods.
Sample Input Code:
class Stack:
pass
Expected Output:
• A functional stack implementation with all required methods and
Docstrings.

Prompt

"Generate a Python Stack class that implements push, pop, peek, and is_empty methods with proper docstrings."

code:

> Users > DELL > Downloads > Untitled-1.py > ...

```python
class Stack:
    """
    Stack implementation using Python list.
    Follows LIFO (Last In First Out) principle.
    """

    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an element to the top of the stack"""
        self.items.append(item)

    def pop(self):
        """Remove and return the top element of the stack"""
        if not self.is_empty():
            return self.items.pop()
        return "Stack is empty"

    def peek(self):
        """Return the top element without removing it"""
        if not self.is_empty():
            return self.items[-1]
        return "Stack is empty"

    def is_empty(self):
        """Check whether the stack is empty"""
        return len(self.items) == 0
```

```python

# Example Usage
s = Stack()

s.push(10)
s.push(20)
s.push(30)

print("Top element:", s.peek())
print("Removed element:", s.pop())
print("Stack empty:", s.is_empty())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users
Top element: 30
Removed element: 30
Stack empty: False
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
```

Justification

push() → Adds an element to the stack.

pop() → Removes the last inserted element.

peek() → Shows the top element without deleting it.

is_empty() → Checks whether the stack contains elements.

Task Description #2 – Queue Implementation
Task: Use AI to implement a Queue using Python lists.
Sample Input Code:
class Queue:
pass
Expected Output:
• FIFO-based queue class with enqueue, dequeue, peek, and size Methods.

Prompt:
Generate a Python Queue class using lists with enqueue, dequeue, peek, and size methods.

code:

C: > Users > DELL > Downloads > Untitled-1.py > Queue > size

```python
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        return self.items.pop(0)

    def peek(self):
        return self.items[0]

    def size(self):
        return len(self.items)
```

Justification

enqueue() → Adds an element to the queue.

dequeue() → Removes the first inserted element.

peek() → Returns the front element without removing it.

size() → Returns the total number of elements in the queue.

Task Description #3 – Linked List
Task: Use AI to generate a Singly Linked List with insert and display methods.
Sample Input Code:
class Node:
pass
class LinkedList:
pass
Expected Output:
• A working linked list implementation with clear method Documentation.

Prompt:
Generate a Python **Singly Linked List** with **insert** and **display** methods.

code:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            self.new_method(temp)

    def new_method(self, temp):
        temp = temp.next
```

Justification

A **Singly Linked List** stores data in nodes where each node points to the **next node** in the list.

Task Description #4 – Hash Table
Task: Use AI to implement a hash table with basic insert, search, and delete methods.
Sample Input Code:
class HashTable:

pass
Expected Output:
• Collision handling using chaining, with well-commented methods.

Prompt:
Generate a Python **Hash Table** with **insert, search, and delete** methods using **chaining for collision handling**.

code:

C: > Users > DELL > Downloads > Untitled-1.py > HashTable > delete

```python
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return key % self.size

    def insert(self, key):
        index = self.hash_function(key)
        self.table[index].append(key)

    def search(self, key):
        index = self.hash_function(key)
        return key in self.table[index]

    def delete(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            self.table[index].remove(key)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\
```

Justification

A **Hash Table** stores data using a **hash function**. If multiple keys map to the same index, **chaining** stores them in a list at that index.

Task Description #5 – Graph Representation
Task: Use AI to implement a graph using an adjacency list.
Sample Input Code:
class Graph:
pass
Expected Output:
• Graph with methods to add vertices, add edges, and display connections.

Prompt:
Generate a Python **Graph** using an **adjacency list** with methods to **add vertices, add edges, and display connections**.

Code:

: > Users > DELL > Downloads > Untitled-1.py > Graph > display

```python
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, v):
        if v not in self.graph:
            self.graph[v] = []

    def add_edge(self, v1, v2):
        self.graph[v1].append(v2)

    def display(self):
        for vertex in self.graph:
            print(vertex, "->", self.graph[vertex])
```

Justification

A **Graph using adjacency list** stores each vertex with a list of its connected vertices (edges).

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - o Stack
  - o Queue
  - o Priority Queue
  - o Linked List
  - o Binary Search Tree (BST)
  - o Graph
  - o Hash Table
  - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt

"Analyze the Smart Hospital Management System features and choose the most suitable data structure (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque) for each feature. Provide a short justification and implement one feature as a Python program."

Code:

ai13.5 | Release Notes: 1.110.1 | Untitled-1.py

C: > Users > DELL > Downloads > Untitled-1.py > ...

```python
import heapq

class EmergencyQueue:
    def __init__(self):
        self.q = []

    def add_patient(self, name, priority):
        heapq.heappush(self.q, (priority, name))

    def treat_patient(self):
        if self.q:
            print("Treating:", heapq.heappop(self.q)[1])

e = EmergencyQueue()
e.add_patient("John",2)
e.add_patient("Alice",1)
e.treat_patient()
```

Justification

Priority Queue is used for emergency cases.

Patients with higher priority (more critical) are treated first.

The Python heapq module efficiently manages the priority queue.

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

Prompt :

"Select suitable data structures for Smart City Traffic Control System features and implement one feature using Python."

code:

```
ai13.5        Release Notes: 1.110.1        Untitled-1.py
C: > Users > DELL > Downloads > Untitled-1.py > ...
class TrafficQueue:
    def __init__(self):
        self.q = []

    def add_vehicle(self, v):
        self.q.append(v)

    def pass_vehicle(self):
        if self.q:
            print(self.q.pop(0))

t = TrafficQueue()
t.add_vehicle("Car")
t.add_vehicle("Bike")
t.pass_vehicle()t.pass_vehicle()
```

Justification

Queue follows **FIFO**, so vehicles pass the signal in the **same order they arrive**.

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge
An e-commerce company wants to build a Smart Online Shopping System with:
1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are

placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Prompt :

"Select suitable data structures for Smart E-Commerce features and implement one feature using Python."

Code:

ai13.5 | Release Notes: 1.110.1 | Untitled-1.py

C: > Users > DELL > Downloads > Untitled-1.py > ...

```python
class OrderQueue:
    """
    Order processing system using Queue (FIFO).
    """

    def __init__(self):
        self.orders = []

    def add_order(self, order):
        self.orders.append(order)

    def process_order(self):
        if self.orders:
            print("Processing:", self.orders.pop(0))


# Example
o = OrderQueue()
o.add_order("Order1")
o.add_order("Order2")
o.process_order()
```

Justification

Queue ensures ***orders are processed in the same order they are placed (FIFO)****.*